Architecting the Cloud: How We Build a SQL Server PaaS at Scale

Nader Sharara

Who am I?

Microsoft CERTIFIED IT Professional Database Administrator 2008

Microsoft CERTIFIED Solutions Expert Data Platform

aws certified Database SPECIALTY

~15 Years in data: From DBA to Cloud Database Architect.

Built and operated SQL Server platforms across on-prem and cloud.

Currently building SQL Server managed services at STACKIT.

Microsoft & AWS Database Specialist.

Blogging @ sqlspark.com - Squash player - Egyptian based in Berlin.

sqlspark.com

nader-sharara-db

Our Journey Today

A journey for designing and building a **SQL Server PaaS** platform, delivered as **“STACKIT SQL Server Flex”.**

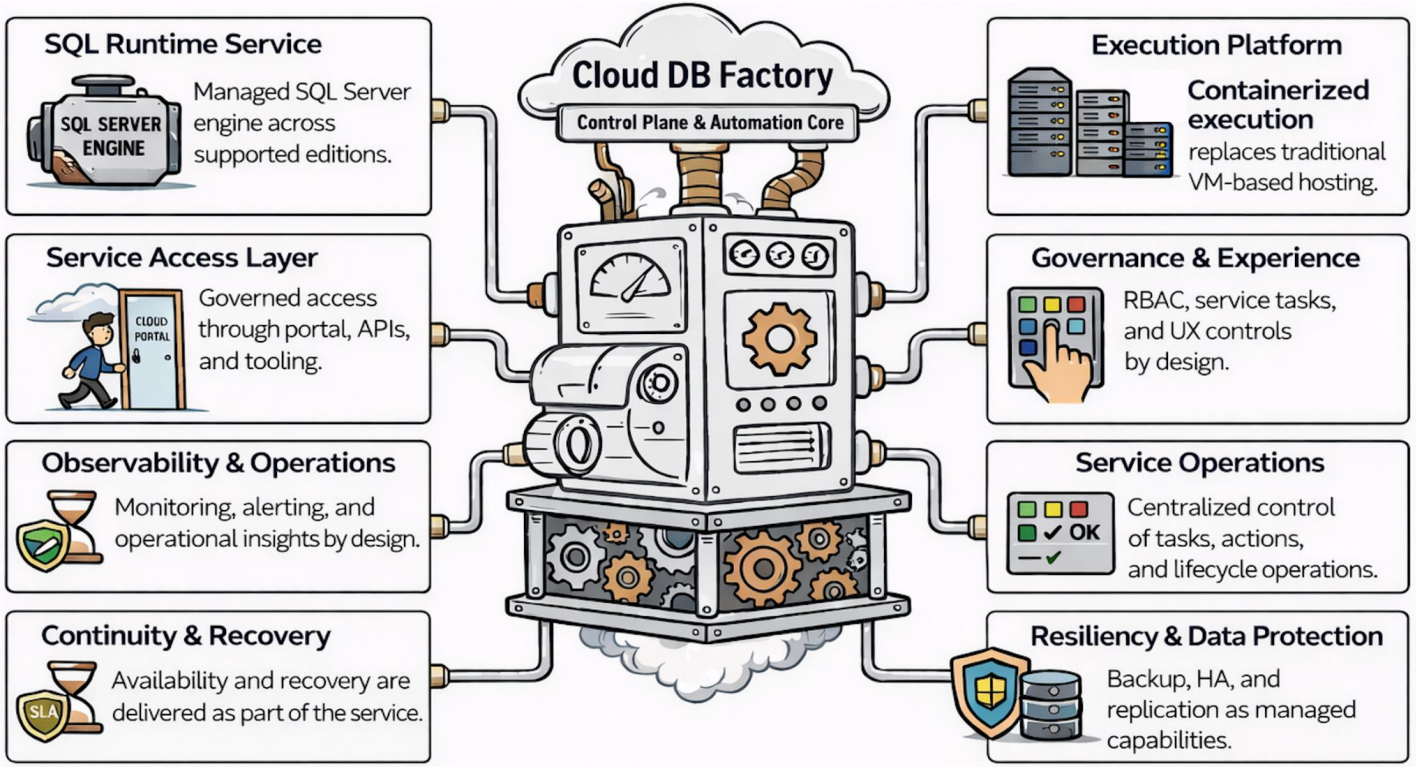
During the next 50 minutes, we will:

- Start with the **architecture** and the foundation of the platform.
- Explore how the SQL Server PaaS platform is **built** and **deployed**.
- Understand how **users interact** with the service to request resources.
- Explain how **security**, **governance**, and **control** are enforced.
- Cover how **high availability** and **disaster recovery** are designed & achieved.
- Show how the platform is **observed** and operated at scale.
- Conclude with the **key lessons** learned along the journey.

For each section, we will focus on the **architectural decisions** and engineering mindset behind its design.

The SQL PaaS Master Blueprint

How do we architect a scalable and manageable SQL Server PaaS platform?



SQL Server PaaS Hosting - Containerization/Kubernetes

How do we design a reliable SQL Server PaaS infrastructure?

Standardized Runtime

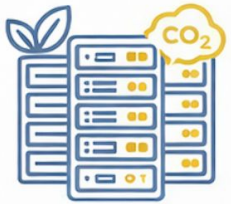
- Golden SQL runtime images
- Consistent configuration baseline
- Reduced drift across tenants

Automated Operations

- Policy-driven lifecycle automation
- Self-healing service behavior
- Safer patching with less downtime

Fast Provisioning

- Fast instance provisioning
- Repeatable deployment workfow
- Scalable service onboarding



Efficient Platform Utilization

- Efficient workload placement
- Better infrastructure utilization
- Platform-aligned sovereignty goals

SQL Server PaaS Deployment – The Platform Build Pipeline

How do we design and build a healthy, production-ready SQL Server PaaS?

1. Infrastructure Foundation

- Validate cluster readiness.
- Provision resources. (storage, network ., etc)

2. Instance Initialization

- Deploy SQL Server instances.
- Execute initialization scripts.

3. Integration & State Definition

- Apply HA settings.
- Expose SQL service via ingress.

Service Accessibility: Customer Provisioning Workflow

How are SQL instances provisioned and made ready?

1. The Service Request (Portal/API)

- Customer selects SQL Instance 'Flavor' (vCPU, RAM, Storage).
- Choose Region, Zone, and SQL Version.
- Submit via Cloud Portal or API.

Request Data

PORTAL

2. Platform-Orchestrated Provisioning

- Defines desired SQL instance state.
- Provisions SQL Server instances.
- Applies resource limits, network policies, and storage quotas.

Instance is Ready!

3. User Autonomy & Setup

- Access via Instance Dashboard or CLI.
- Create first SQL Admin User.
- Configure initial Databases & Users for application.

Instance Operations: Secure Command & Control Layer

How do we securely control and govern all SQL instance operations?

1. The Management Request

- User triggers action (e.g., 'New Database: Sales_Prod').
- Input values are captured.
- Initial validations performed.

2. The Secure API Gateway (Governance)

- API validates request.
- RBAC & Permission Checks.
- Input is sanitized.

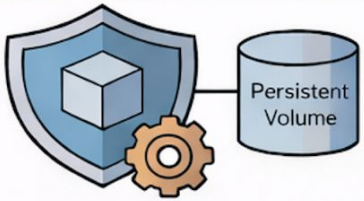
3. Encapsulated Stored Procedure Execution

- API calls dedicated SP in isolated instance.
- No Dynamic SQL, Immune to SQL Injection.

High Availability – Resiliency by Design

How do we deliver the right level of resiliency for different SQL Server workloads?

Tier 1: Standalone (General Purpose)

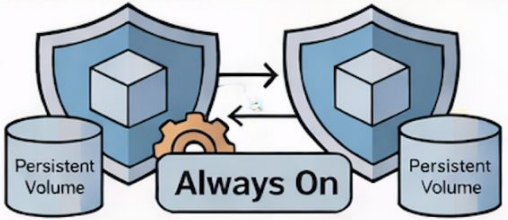


Infrastructure Self-Healing

- Shared Persistent Volume (Network Storage)
- Single Copy (No Redundancy)
- Automated Recovery (Node/Pod Failure)
- Data Protection relies on Backups

A Spectrum of Choice:
Infrastructure-only vs.
Multi-Copy Redundancy

Tier 2: Multi-Node (Business Critical)

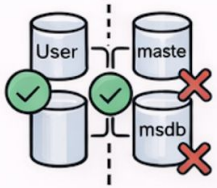


Application-Level Always On

- Dedicated PVC per Replica (Independent Persistence)
- 1-2 extra copies (Synchronized)
- Node, Storage, or SQL Service Failover
- Real-time application redundancy

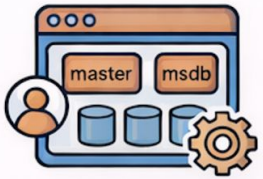
Business Critical Tier: Contained Availability Groups

How do we deliver a fully portable and customer-ready availability group abstraction?



1. Traditional AG Metadata Gap

- Logins and jobs are local to 'master' (not synchronized)
- Metadata does not move with user data
- Failover can result in orphaned accounts



2. Contained Solution (Metadata Portability)

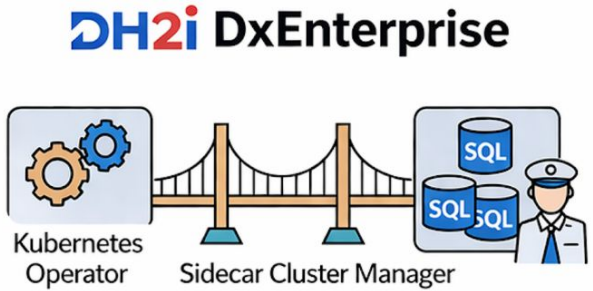
- Logins, jobs, and permissions move with the AG
- System databases are synchronized
- Instance-level identity moves with the workload

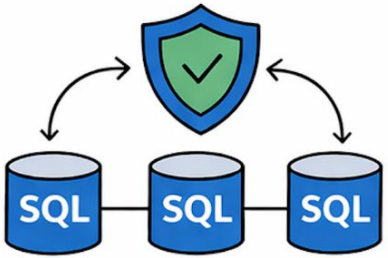
3. Automated Plumbing & simplified Failover

- Endpoints, listeners, and certificates handled automatically
- Rolling updates and health checks managed by the platform
- Failover behavior integrated into the platform design

Business Critical: The Cluster Manager Abstraction

How do we provide a cluster manager abstraction for SQL Server in Kubernetes?





1. Why Kubernetes Alone Is Not Enough

- No native clustering (WSFC equivalent) on Linux
- Traditional clustering (Pacemaker/Corosync) is not cloud-native

2. Sidecar-Based Cluster Manager (DxEnterprise)

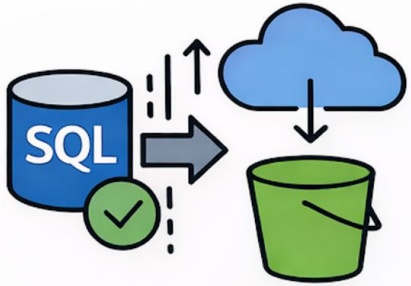
- Lightweight, cloud-native cluster manager
- Enables SQL-aware failover beyond K8s primitives

3. Simplified HA & Cluster Integrity

- AG-aware health monitoring and synchronization
- Eliminates manual cluster management
- Automated multi-zone failover

Disaster Recovery - Recovery by Design

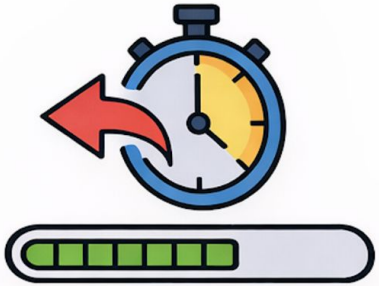
How do we deliver predictable disaster recovery for SQL Server PaaS workloads?



1. Continuous Protection (RPO)

< 15 Minute RPO

- Stream transaction logs directly to S3
- Decoupled storage
- Automated Full/Diff/Log scheduling



2. Automated Recovery (RTO)

< 4 Hour RTO

- Point-in-Time Recovery (PITR) to exact second
- Handles complex restore chain
- Non-disruptive restore

3. Isolated Recovery Vault

- Built-in local redundancy
- Isolated durable storage
- Encrypted & isolated from compute
- Data remains within isolation boundary

Security & Governance - Controlled Access by Design

How do we secure SQL Server access without exposing the underlying engine?

The Risk of Direct Access

- Direct SQL access enables unsafe operations
- Dynamic SQL increases injection risk
- Sysadmin privileges break managed service boundaries

Platform-Controlled Access Model

- All operations go through platform API layer
- Execution via encapsulated stored procedures
- RBAC-based access (no sysadmin access)
- No direct interaction with the database engine

Secure & Governed Operations

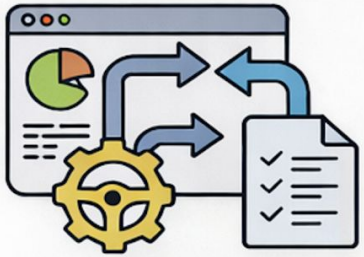
- SQL injection eliminated by design
- Customer Admin role (safe operational scope)
- Encryption at rest and in transit enforced
- Platform integrity preserved across operations

Separation of Concerns – Control, Compute, and State

How do we separate control, compute, and data in a SQL Server PaaS platform?

Control Plane

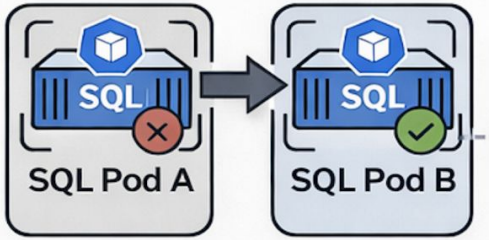
Governance & Lifecycle



- ✓ Operator-driven platform **intelligence**
- ✓ API & policy-enforced guardrails
- ✓ Centralized lifecycle management

Application

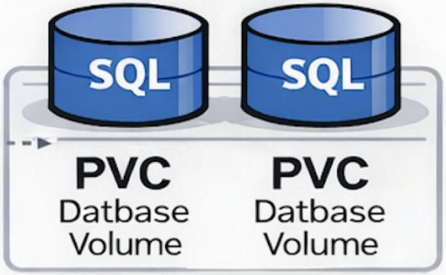
SQL Runtime



- ✓ **Replaceable, stateless compute layer**
- ✓ Isolated from persistent state
- ✓ Immutable runtime instances

3. Data Layer

Persistent State

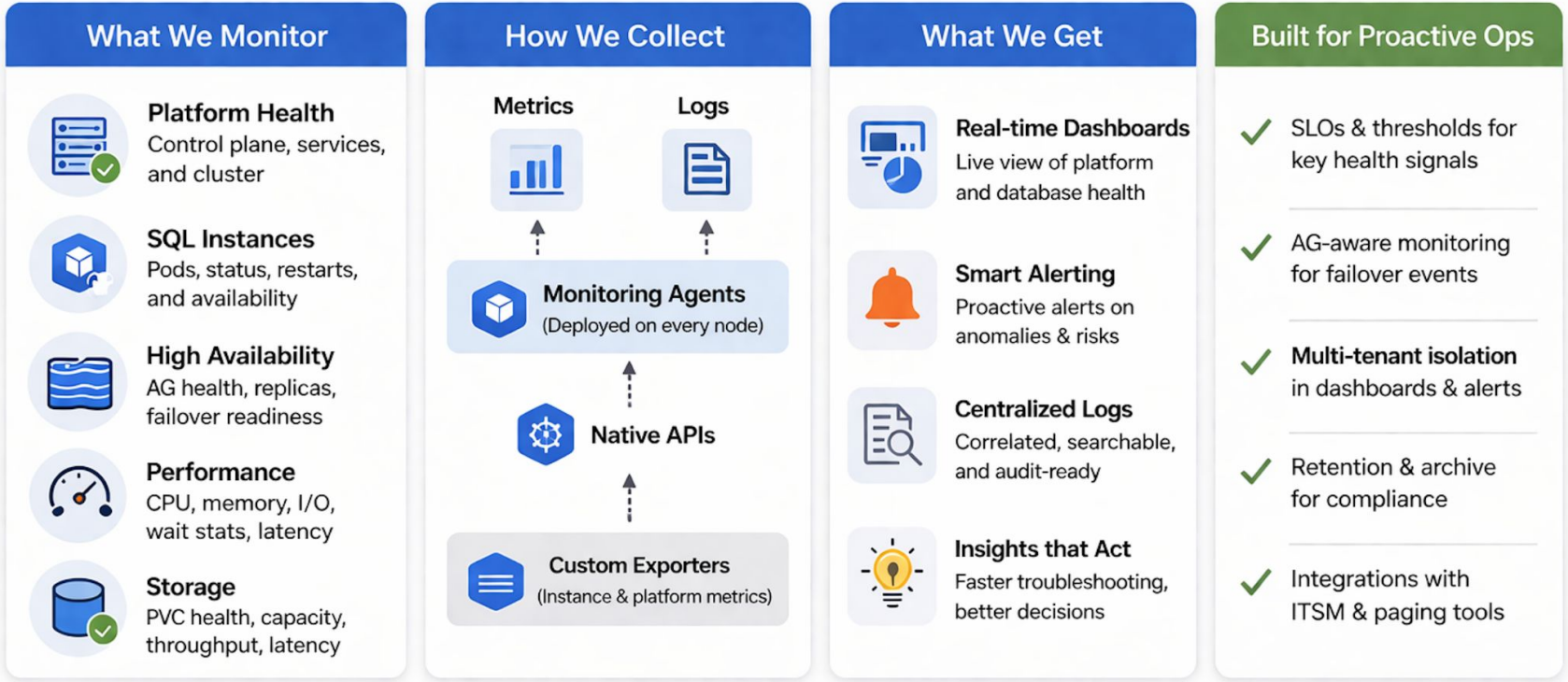


Decoupled storage ensures durability.

- ✓ Durable storage decoupled from pods
- ✓ State persistent across compute changes

Observability by Design – End-to-End Visibility Across the Platform

How do we provide unified visibility across the entire SQL Server PaaS platform?



Lessons Learned - Building a SQL Server PaaS in Practice

What we really learned during this journey?

- **Sovereignty** must be part of the design — not an afterthought.
- **Architecture decisions** show up during operations — not during design.
- **Separation of concerns** is not optional — it simplifies operations.
- **Security** must be designed from day one — not added later.
- **Observability** gaps reveal themselves during real incidents.
- **Automation** is not optional — it is required to scale.

The hardest part was not building the system — but operating it reliably.

Questions

Feedback

Architecting the Cloud: How We Build a SQL Server PaaS at Scale